



Get started Prepare ▾

Get full access



Blog Menu

Series Contents

Top 30 React Interview Questions and Answers to Get Hired in 2025

30 essential React interview questions and answers to help you prepare for front-end job interviews in 2025



Starter

[#react](#)[#interviews](#)

GreatFrontEnd Team

17 min read Jan 29, 2025

Copy link

Share



Top 30 React Interview Questions and Answers to Get Hired in 2025

As the demand for skilled front-end developers rises in 2025, proficiency in React has become essential. This JavaScript library is widely used for building dynamic user interfaces, and mastering it requires a solid understanding of its core principles and advanced features.

To help you prepare for your upcoming interviews, we've compiled 30 essential React interview questions and answers that cover a range of topics from fundamental concepts to advanced techniques. Whether you're an experienced developer or new to React, this guide will equip you with the knowledge needed to excel in interviews.

If you're looking for more in-depth React interview preparation materials, also check out these resources:

- [React Interview Playbook](#)
- [Practice React coding interview questions](#)
- [Practice React quiz interview questions](#)

1. What is the difference between React Node, Element, and Component?

React Node, Element, and Component are three fundamental concepts in React.

- **Node:** A React Node is any renderable unit in React, like an element, string, number, or `null`.
- **Element:** An element is a plain object that represents a DOM element or a component. It describes what you want to see on the screen. Elements are immutable and are used to create React components.
- **Component:** A component is a reusable piece of UI that can contain one or more elements. Components can be either functional or class-based. They accept inputs called props and return React elements that describe what should appear on the screen.

[Read more about it here](#)

2. What are React Fragments used for?

React Fragments are a feature introduced in React 16.2 that allows you to group multiple children elements without adding extra nodes to the DOM. Fragments are useful when you need to return multiple elements from a component but don't want to wrap them in a parent element. They help keep the DOM structure clean and avoid unnecessary div wrappers.

Here's an example of using React Fragments:

```
1  function App() {  
2    return (  
3      <>  
4        <Header />  
5        <Main />  
6        <Footer />  
7      </>  
8    );  
9  }
```

In this example, the `<>...</>` syntax is a shorthand for declaring a React Fragment. It allows you to group the `Header`, `Main`, and `Footer` components without adding an extra `div` to the DOM.

[Read more about it here](#)

3. What is the purpose of the `key` prop in React?

The `key` prop is a special attribute used in React to uniquely identify elements in a list. When rendering a list of elements, React uses the `key` prop to keep track of each element's identity and optimize the rendering process. The `key` prop helps React identify which items have changed, been added, or been removed, allowing it to update the DOM efficiently.

[Read more about it here](#)

4. What is the consequence of using array indices as keys in React?

Using array indices as keys in React can lead to performance issues and unexpected behavior. When you use array indices as keys, React uses the index to identify elements and track changes. However, this approach can cause problems when the array is modified, as React may not be able to differentiate between elements with the same key.

For example, consider the following code:

```
1  function App() {
2    const items = ['A', 'B', 'C'];
3
4    return (
5      <ul>
6        {items.map((item, index) => (
7          <li key={index}>{item}</li>
8        ))}
9      </ul>
```

```
10    );  
11  }
```

If you add or remove items from the `items` array, React may not update the DOM correctly because it relies on the index as the key. To avoid this issue, it's recommended to use unique IDs or keys that are stable across renders.

[Read more about it here](#)

5. What is the difference between Controlled and Uncontrolled React components?

Controlled and Uncontrolled components are two common patterns used in React to manage form inputs and state.

- **Controlled Components:** In a controlled component, form data is handled by React state and is updated via state changes. The input value is controlled by React, and any changes to the input are handled by React event handlers. Controlled components provide more control over form inputs and allow you to validate and manipulate the input data before updating the state.
- **Uncontrolled Components:** In an uncontrolled component, form data is handled by the DOM itself, and React does not control the input value. The input value is managed by the DOM, and you can access the input value using a ref. Uncontrolled components are useful for integrating with third-party libraries or when you need to access the input value imperatively.

Example of controlled component:

```
1  function ControlledInput() {  
2    const [value, setValue] = React.useState('');  
3    return (  
4      <input  
5        type="text"  
6        value={value}
```

```
7      onChange={e => setValue(e.target.value)}
8    />
9  );
10 }
```

Example of uncontrolled component:

```
function UncontrolledInput() {
  const inputRef = React.useRef();
  return <input type="text" ref={inputRef} />;
}
```

[Read more about it here](#)

6. How would you lift the state up in a React application, and why is it necessary?

Lifting state up in React involves moving the state from child components to their nearest common ancestor. This pattern is used to share state between components that don't have a direct parent-child relationship. By lifting state up, you can avoid prop drilling and simplify the management of shared data. Example:

```
1  const Parent = () => {
2    const [counter, setCounter] = useState(0);
3
4    return (
5      <div>
6        <Child1 counter={counter} />
7        <Child2 setCounter={setCounter} />
8      </div>
9    );
10 };
11
12 const Child1 = ({ counter }) => <h1>{counter}</h1>;
13 const Child2 = ({ setCounter }) => (
```

```
14   <button onClick={() => setCounter((prev) => prev + 1)}>Increment</button>  
15   );
```

7. What are Pure Components?

Pure Components are a type of React component that extends `React.PureComponent` or uses the `React.memo` higher-order component. Pure Components are optimized for performance and implement a `shouldComponentUpdate` method that performs a shallow comparison of props and state to determine if the component should re-render. If the props and state of a Pure Component have not changed, React skips the re-rendering process, improving performance.

Pure Components are useful when you have components that render the same output given the same input and don't rely on external state or side effects. By using Pure Components, you can prevent unnecessary re-renders and optimize your React application.

8. What is the difference between `createElement` and `cloneElement` ?

- `createElement` : Used to create a new React element by specifying its type (e.g., 'div', a React component), props, and children.

```
React.createElement('div', { className: 'container' }, 'Hello World');
```

- `cloneElement` : Used to clone an existing React element and optionally modify its props while keeping the original element's children and state.

```
const element = <button className="btn">Click Me</button>;  
const clonedElement = React.cloneElement(element, { className: 'btn-primary' });
```

9. What is the role of PropTypes in React?

PropTypes is a library used in React to validate the props passed to a component. PropTypes help you define the types of props a component expects and provide warnings in the console if the props are of the wrong type. PropTypes are useful for documenting component APIs, catching bugs early, and ensuring that components receive the correct data.

Here's an example of using PropTypes:

```
1  import PropTypes from 'prop-types';
2
3  const Greeting = ({ name }) => <h1>Hello, {name}!</h1>;
4
5  Greeting.propTypes = {
6    name: PropTypes.string.isRequired,
7  };
```

In this example, the `Greeting` component expects a prop named `name` of type `string`. If the `name` prop is not provided or is not a string, a warning will be displayed in the console.

10. What are stateless components?

Stateless components do not manage internal state; they receive data via props and focus solely on rendering UI based on that data.

Example:

```
function StatelessComponent({ message }) {
  return <div>{message}</div>;
}
```

11. What are stateful components?

Stateful components manage their own internal state and can update their UI based on user interactions or other events.

Example:

```
1  function StatefulComponent() {  
2    const [count, setCount] = React.useState(0);  
3  
4    return (  
5      <div>  
6        <p>{count}</p>  
7        <button onClick={() => setCount(count + 1)}>Increment</button>  
8      </div>  
9    );  
10 }
```

12. What are the benefits of using hooks in React?

Hooks allow you to use state and other React features in functional components, eliminating the need for classes. They simplify code by reducing reliance on lifecycle methods, improve code readability, and make it easier to reuse stateful logic across components. Common hooks like `useState` and `useEffect` help manage state and side effects.

[Read more about it here](#)

13. What are the rules of React hooks?

React hooks follow a set of rules to ensure they are used correctly:

- **Only call hooks at the top level:** Hooks should only be called from the top level of a functional component or from custom hooks. They should not be called inside loops, conditions, or nested functions.

- **Only call hooks from React functions:** Hooks should only be called from React components or custom hooks. They should not be called from regular JavaScript functions.
- **Use hooks in the same order:** Hooks should be called in the same order on every render to ensure the component's state is consistent.
- **Don't call hooks conditionally:** Hooks should not be called conditionally based on a condition. They should always be called in the same order on every render.

[Read more about it here](#)

14. What is the difference between `useEffect` and `useLayoutEffect` in React?

- `useEffect` : Runs after the browser has painted the screen and is used for side effects that don't block the browser's painting process. It's asynchronous and runs after the render is committed to the DOM.
- `useLayoutEffect` : Runs synchronously after the DOM has been updated but before the browser has painted the screen. It's used for side effects that require the DOM to be updated synchronously.

In most cases, you should use `useEffect` unless you need to perform a synchronous side effect that requires the DOM to be updated immediately.

Example:

```
1  import React, { useEffect, useLayoutEffect, useRef } from 'react';
2
3  function Example() {
4    const ref = useRef();
5
6    useEffect(() => {
7      console.log('useEffect: Runs after DOM paint');
8    });
```

```
9
10  useEffect(() => {
11    console.log('useLayoutEffect: Runs before DOM paint');
12    console.log('Element width:', ref.current.offsetWidth);
13  });
14
15  return <div ref={ref}>Hello</div>;
16 }
```

[Read more about it here](#)

15. What does the dependency array of `useEffect` affect?

The dependency array of `useEffect` specifies the values that the effect depends on. When the values in the dependency array change, the effect is re-run. If the dependency array is empty, the effect runs only once after the initial render.

Example:

```
useEffect(() => {
  // Effect code
}, [value1, value2]);
```

In this example, the effect will run whenever `value1` or `value2` changes. If the dependency array is omitted (`[]`), the effect will run only once after the initial render.

[Read more about it here](#)

16. What is the `useRef` hook in React and when should it be used?

The `useRef` hook in React creates a mutable reference that persists across renders. It can be used to store references to DOM elements, manage focus, or store mutable values that don't trigger re-renders.

Example:

```
1  function TextInputWithFocusButton() {
2    const inputRef = useRef(null);
3
4    const handleClick = () => {
5      inputRef.current.focus();
6    };
7
8    return (
9      <>
10       <input ref={inputRef} type="text" />
11       <button onClick={handleClick}>Focus Input</button>
12     </>
13   );
14 }
```

In this example, the `inputRef` is used to store a reference to the input element, and the `handleClick` function focuses the input when the button is clicked.

[Read more about it here](#)

17. Why does React recommend against mutating state?

React recommends against mutating state directly because it can lead to unexpected behavior and bugs. When you mutate state directly, React may not detect the changes, causing the UI to become out of sync with the application's state. To ensure that React detects state changes correctly, you should always update state using the `setState` function or hooks like `useState`.

[Read more about it here](#)

18. What is reconciliation in React?

Reconciliation is the process by which React updates the DOM to match the virtual DOM after a component's state or props change. React compares the previous virtual DOM with the new virtual DOM and determines the minimum number of changes needed to update the DOM efficiently. Reconciliation is an essential part of React's performance optimization strategy and helps minimize the number of DOM manipulations required.

[Read more about it here](#)

19. What is hydration in React?

Hydration is the process by which React attaches event listeners and updates the DOM to match the virtual DOM on the client side. Hydration is necessary for server-side rendered React applications to ensure that the client-side rendered content is interactive and matches the server-rendered content. During hydration, React reconciles the server-rendered HTML with the client-side virtual DOM and updates the DOM to match the virtual DOM structure.

[Read more about it here](#)

20. Explain higher-order components (HOCs).

Higher-order components (HOCs) are functions that take a component as an argument and return a new component with enhanced functionality. HOCs are used to share code between components, add additional props or behavior to components, and abstract common logic into reusable functions.

Example:

```
1  const withLogger = (WrappedComponent) => {  
2    return (props) => {  
3      console.log('Component rendered:', WrappedComponent.name);  
4      return <WrappedComponent {...props} />;  
5    };  
}
```

```
6   };  
7  
8   const EnhancedComponent = withLogger(MyComponent);
```

In this example, the `withLogger` HOC logs the name of the component every time it renders. The `EnhancedComponent` is a new component that includes the logging functionality.

[Read more about it here](#)

21. What are some common performance optimization techniques in React?

Some common performance optimization techniques in React include:

- **Memoization:** Use memoization techniques like `useMemo` and `useCallback` to cache expensive computations and prevent unnecessary re-renders.
- **Code Splitting:** Split your code into smaller chunks and load them dynamically to reduce the initial bundle size and improve loading times.
- **Lazy Loading:** Use lazy loading to load components or resources only when they are needed, reducing the initial load time of your application.
- **Virtualization:** Implement virtualization techniques like windowing or infinite scrolling to render only the visible elements in long lists or tables, improving performance.
- **Server-Side Rendering:** Use server-side rendering to pre-render your React components on the server and send the HTML to the client, reducing the time to first paint.

22. What is the purpose of the `useReducer` hook in React?

The `useReducer` hook in React is used to manage complex state logic in functional components. It is an alternative to `useState` and allows you to update state based on the previous state and an action. `useReducer` is useful for managing state transitions that depend on the current state and require more complex logic than simple updates.

Example:

```
1  const initialState = { count: 0 };
2
3  const reducer = (state, action) => {
4    switch (action.type) {
5      case 'increment':
6        return { count: state.count + 1 };
7      case 'decrement':
8        return { count: state.count - 1 };
9      default:
10       return state;
11    }
12  };
13
14  const Counter = () => {
15    const [state, dispatch] = useReducer(reducer, initialState);
16
17    return (
18      <div>
19        Count: {state.count}
20        <button onClick={() => dispatch({ type: 'increment' })}>Increment</button>
21        <button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>
22      </div>
23    );
24  };
```

In this example, the `useReducer` hook is used to manage the state of a counter component based on different actions.

[Read more about it here](#)

23. How do you test a React application?

Testing React applications can be done using Jest and React Testing Library. Jest serves as the testing framework while React Testing Library provides utilities for testing components similarly to user interactions.

[Read more about it here](#)

24. What is server-side rendering (SSR)?

Server-side rendering (SSR) is a technique used to pre-render React components on the server and send the HTML to the client. SSR improves the performance of React applications by reducing the time to first paint and making the content accessible to search engines and users with slow internet connections.

[Read more about it here](#)

25. What is Static site generation (SSG)?

Static site generation (SSG) is a technique used to pre-render static HTML pages at build time. SSG generates static HTML files for each page of a website, which can be served directly to users without the need for server-side rendering. SSG improves performance, reduces server load, and simplifies hosting and deployment.

[Read more about it here](#)

26. What is lazy loading in React?

Lazy loading is a technique used to load components or resources only when they are needed. Lazy loading helps reduce the initial load time of your application by deferring the loading of non-essential components until they are required. React provides a `React.lazy` function and `Suspense` component to implement lazy loading in your application.

27. What is the purpose of the `useContext` hook in React?

The `useContext` hook in React is used to access the value of a context provider in a functional component. It allows you to consume context values without using a consumer component.

`useContext` is useful for accessing global data or settings in your application without passing props down the component tree.

Example:

```
1  const ThemeContext = React.createContext('light');
2
3  const ThemeProvider = ({ children }) => {
4    return <ThemeContext.Provider value="dark">{children}</ThemeContext.Provider>;
5  };
6
7  const ThemeConsumer = () => {
8    const theme = React.useContext(ThemeContext);
9    return <div>Theme: {theme}</div>;
10  };
```

In this example, the `ThemeConsumer` component uses the `useContext` hook to access the value of the `ThemeContext` provider.

28. What is the purpose of the `useMemo` hook in React?

The `useMemo` hook in React is used to memoize expensive computations and cache the result to prevent unnecessary re-renders. `useMemo` takes a function and an array of dependencies and returns the memoized value. The memoized value is recalculated only when the dependencies change.

Example:

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

In this example, the `computeExpensiveValue` function is memoized, and the result is cached until the `a` or `b` dependencies change.

[Read more about it here](#)

29. What is the purpose of the `useCallback` hook in React?

The `useCallback` hook in React is used to memoize callback functions and prevent unnecessary re-renders of components that depend on those callbacks. `useCallback` takes a function and an array of dependencies and returns a memoized version of the function. The memoized function is only recalculated when the dependencies change.

Example:

```
const memoizedCallback = useCallback(() => {  
  doSomething(a, b);  
}, [a, b]);
```

In this example, the `doSomething` function is memoized, and the memoized callback is cached until the `a` or `b` dependencies change.

[Read more about it here](#)

30. What is the purpose of the `useImperativeHandle` hook in React?

The `useImperativeHandle` hook in React is used to customize the instance value that is exposed to parent components when using `React.forwardRef`. It allows you to define which properties or methods of a child component's instance should be accessible to parent components when using a ref.

Example:

```
1  const ChildComponent = React.forwardRef((props, ref) => {
2    const inputRef = useRef(null);
3
4    useImperativeHandle(ref, () => ({
5      focus: () => {
6        inputRef.current.focus();
7      },
8    }));
9
10   return <input ref={inputRef} />;
11 });
12
13 const ParentComponent = () => {
14   const childRef = useRef(null);
15
16   const handleClick = () => {
17     childRef.current.focus();
18   };
19
20   return (
21     <>
22       <ChildComponent ref={childRef} />
23       <button onClick={handleClick}>Focus Input</button>
24     </>
25   );
26 };
```

In this example, the `useImperativeHandle` hook is used to expose the `focus` method of the `inputRef` to the parent component when using a `ref`.

If you'd like to explore more React interview problems beyond these 30, check out our [Top ReactJS Interview Questions repository](https://www.greatfrontend.com/blog/top-30-reactjs-interview-questions-and-answers) - a curated list of around 50 real-world questions sourced from actual interview experiences.

[Next →](#)

10+ Must-know JavaScript Coding Interview Questions

Related articles

30 Basic to Advanced React Interview Questions with Solutions

30 Basic to Advanced React Interview Questions with Solutions

Jul 1, 2025 [#react](#)

30 React interview questions and solutions, covering basic to advanced topics. Ideal for developers preparing for their next job interview in 2025

50 React.js Interview Questions for Experienced Developers

50 React.js Interview Questions for Experienced Developers

Jun 22, 2025 [#react](#)

50 React JS interview questions for experienced developers. Explore detailed solutions to enhance your preparation for 2025 job interviews



Subscribe to our newsletter

Notify me

Practice

[Get started](#)

[JavaScript functions](#)

[User interface coding](#)

[System design](#)

[Quiz](#)

Guides

[Front End Interview Playbook](#)

[Front End System Design Playbook](#)

[React Interview Playbook](#)

[Behavioral Interview Playbook](#)

Study Plans

[1 Week Plan](#)

[1 Month Plan](#)

[3 Months Plan](#)

Company

[Pricing](#)

[Promotions](#)

[Roadmap](#)

[About](#)

[Team](#)

[Contact us](#)

[Advertise with us](#) New

[Become an affiliate](#)

[Careers](#)

[Blog](#)

[Medium](#)

[DEV Community](#)



English (US) ▾

